

TECNOLOGÍAS DE LA INFORMACIÓN EN LINEA

SISTEMAS EMBEBIDOS

3 créditos



Profesor:

Ing. Freddy Carrera Sanchez, Msc

Titulaciones	Semestre
Ingeniero en Tecnologías de la Información	Sexto

Tutorías: El profesor asignado se publicará en el entorno virtual de aprendizaje (online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección **CAFETERÍA VIRTUAL**.

PERÍODO ABRIL – AGOSTO 2026

Índice

Tabla de contenido

Unidad 3. EL SOFTWARE DE UN SISTEMA EMBEBIDO	4
Tema 3.1: Software de Sistemas Embebidos	5
3.1.1 Desarrollo del Software embebido.....	5
Aspectos importantes.....	5
Arquitectura y modelos del software.	6
Diseño con diagramas de flujo.	8
Programación del software.	12
3.1.2 Integración del Software embebido	13
El proceso de construcción (<i>Build</i>).	13
Ejemplo del proceso de integración con Arduino IDE.	17
Tema 3.2: Sistemas operativos para dispositivos embebidos	21
3.2.1 Sistemas operativos embebidos.....	21
Sistemas operativos en tiempo real	22
Caso de estudio de un RTOS: RIOT.	23
3.2.2 Drivers y mecanismos de interrupción	25
Fundamentos de interrupciones.	25
Implementación de Interrupciones.	26
3.2.3 Depuración del SW embebido.....	29
Herramientas & Software de la asignatura	31
Bibliografía.....	32

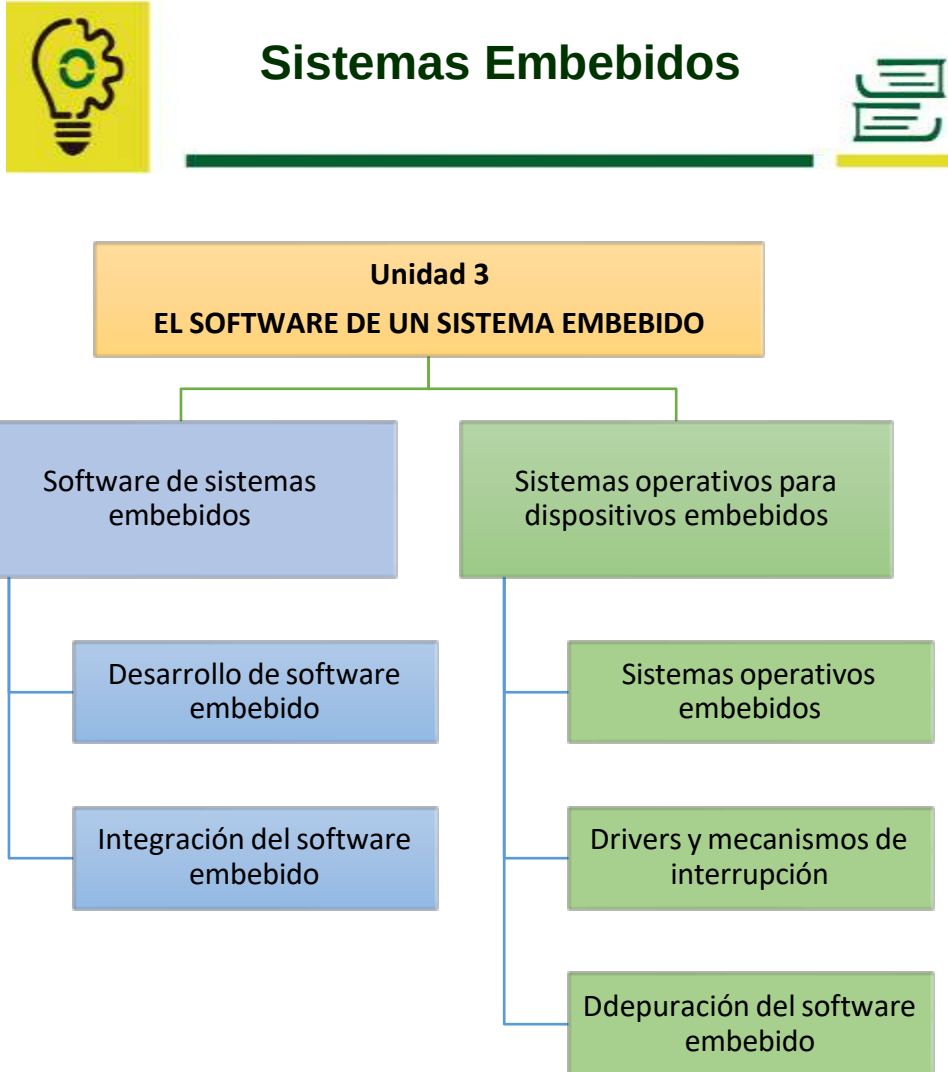


Organización de la lectura para el estudiante por semana del compendio

Semanas	Paginas
Semana 9	Páginas 4 – 19
Semana 10	Páginas 20 – 27
Semana 11	Páginas 28 – 30
Semana 12	Repaso

Resultado de aprendizaje de la asignatura

Implementar un sistema embebido básico mediante el uso de herramientas de prototipado de hardware para el control de dispositivos que soporten soluciones en diferentes ámbitos.



Unidad 3. EL SOFTWARE DE UN SISTEMA EMBEBIDO

Resultado de aprendizaje de la unidad: Utilizar interfaces de software/hardware simples usando lenguajes de programación de alto nivel para la interacción eficaz y segura con sensores y actuadores.

Tema 3.1: Software de Sistemas Embebidos



3.1.1 Desarrollo del Software embebido

Aspectos importantes.

El software de un sistema embebido debe tomar en cuenta algunos factores críticos para su desarrollo. Entre ellos se tienen:

- Limitado espacio de memoria.
- CPUs restringidas en reloj (frecuencia) y longitud de palabra (tamaño en bits).
- Fuertes restricciones al consumo de energía.
- Conectividad limitada/esporádica.
- Dimensiones físicas acotadas.
- Tiempo real.
- Aplicaciones críticas.

Cuando se desarrolla software para un sistema embebido es fundamental considerar los siguientes aspectos:

- **Conocimiento de Hardware:** Hay que familiarizarse con los circuitos integrados y su funcionamiento.
- **Código eficiente:** El software debe estar optimizado, cada línea de código cuenta.
- **Interfaces de periféricos:** Se debe tener claro cómo configurar los periféricos para acceder a ellos o enviarles datos.
- **Código Robusto:** Tomar en cuenta que el código debe ejecutarse mucho tiempo.
- **Recursos mínimos:** Saber administrarlos adecuadamente. Recordar que una de las principales funciones del software es la administración y control de los recursos de hardware:
 - Administración del procesador.
 - Administración de dispositivos de entrada/salida.
 - Administración de la memoria.
 - Administración del almacenamiento persistente.
- **Código reusable:** Código siempre puede ser portable a otros proyectos.

- **Herramientas de desarrollo:** El desarrollador debe poderse adaptar a los distintos entornos existentes, y decidir cuál usar de acuerdo a las necesidades, disponibilidad, e incluso gustos (Barr & Massa, 2013).

Arquitectura y modelos del software.

De acuerdo a los factores mencionados, y a los requisitos del sistema, se podría tener una arquitectura básica o compleja:

- Una arquitectura básica es más simple, utiliza mejor los recursos de hardware, pero quizás requiera más código.
 - Es Ideal para sistemas embebidos pequeños.
- Una arquitectura compleja permite obtener sistemas con más funciones, utilizando menos código, pero requiere hardware con más recursos.
 - Más utilizado en sistemas embebidos de altas prestaciones.

Un ejemplo de estas dos arquitecturas se puede apreciar en la Figura 1, en donde se representan los bloques que componen el sistema en ambos casos. Los dos esquemas tienen bloques similares, por ejemplo el hardware es el mismo en ambos diagramas.

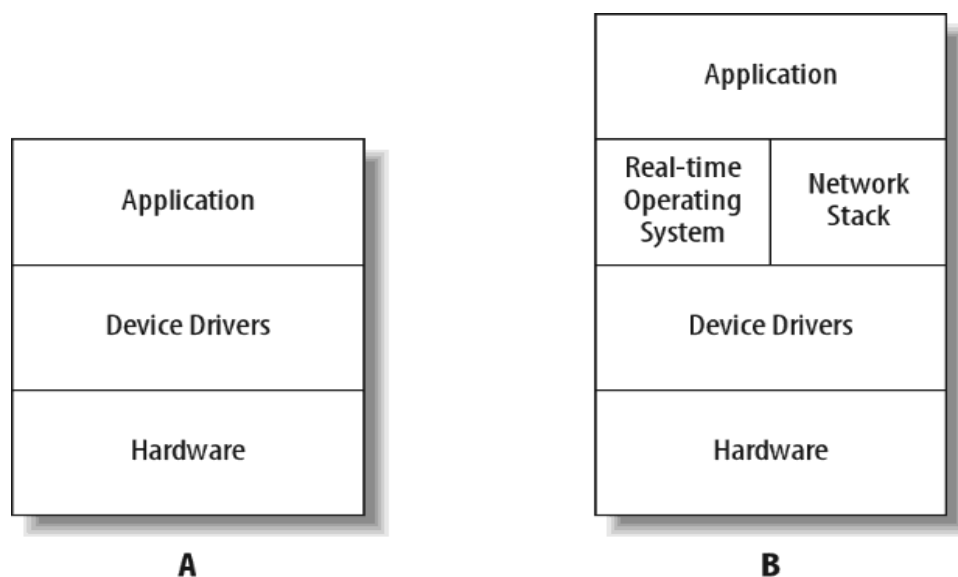


Figura 1. (A) Arquitectura básica de software para un sistema embebido; (B) Arquitectura compleja del software para un sistema embebido (Barr & Massa, 2013).

Los controladores de dispositivos (*drivers*) son módulos de software incorporados que contienen la funcionalidad para operar los dispositivos de hardware individuales. El motivo del software del controlador del dispositivo es eliminar la necesidad de que la aplicación sepa cómo controlar cada pieza de hardware. Normalmente, cada controlador

de dispositivo individual solo necesitaría saber cómo controlar su dispositivo de hardware. Por ejemplo, para un horno de microondas, controladores de dispositivos separados controlan el teclado, la pantalla, la sonda de temperatura y el control de radiación. Los fabricantes de componentes suelen facilitar librerías que contienen estos drivers.

Si se requiere más funcionalidad, a veces es necesario incluir capas adicionales en el software embebido para ayudar con esta funcionalidad adicional. En este ejemplo, el diagrama complejo incluye un Sistema Operativo en Tiempo Real (RTOS) y una pila de redes. El RTOS puede ayudar al programador a separar la funcionalidad de la aplicación en distintas tareas para una mejor organización del software de la aplicación y un sistema con mayor capacidad de respuesta. La pila de red también se suma a la funcionalidad del sistema embebido, por ejemplo para poder enviar mensajes a otros dispositivos en una red.

Las responsabilidades de la capa de software de aplicación son las mismas en los diagramas de software embebido básico y complejo. En un horno de microondas, la aplicación procesa las diferentes entradas y controla las salidas en función de lo que el usuario ordena que haga (Barr & Massa, 2013).

Con esta base se puede plantear un modelo de software embebido basado en capas. Esta una concepción arquitectural del desarrollo de software donde la complejidad se subdivide en conjuntos separados o capas (Figura 2).

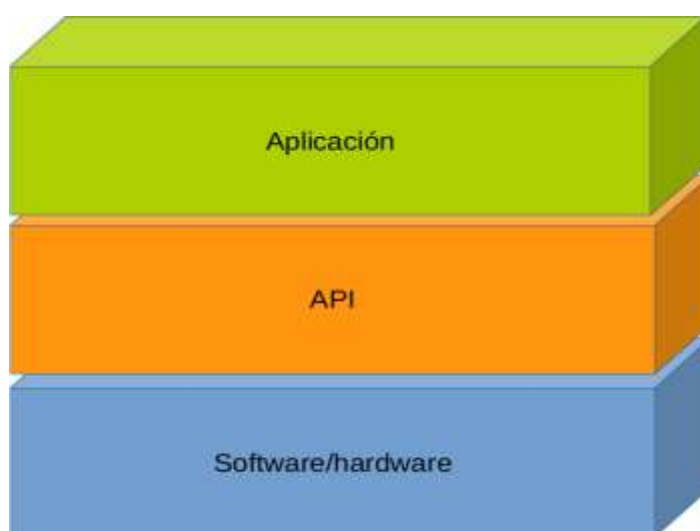


Figura 2. Ejemplo de un Modelo de capas.

En este modelo cada capa tiene una serie de responsabilidades hacia la capa superior, y recibe una serie de servicios de la capa inferior. Es decir, cada capa ofrece y utiliza servicios completamente especificados, tal como se ve en la Figura 3.

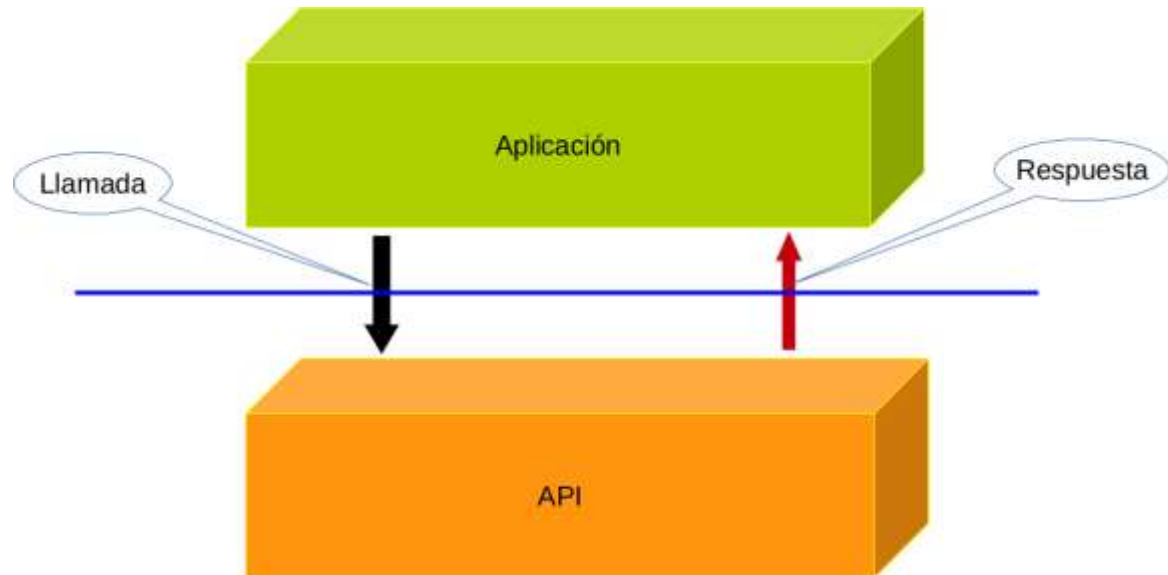


Figura 3. Esquema de llamada a servicios de una API.

Un API puede implementarse de varias maneras para poder ser utilizada en aplicaciones, por ejemplo:

- **Basada a funciones:** Se presenta como una librería que se carga dinámica o estáticamente, y un conjunto de funciones que pueden invocarse desde un programa.
- **Basada en eventos:** Utilizadas para comunicaciones asíncronas con dispositivos de externos (como sensores). La interacción se lleva a cabo mediante “funciones de call-back”

Diseño con diagramas de flujo.

Los diagramas de flujo son una de las herramientas más antiguas y populares que se utilizan para expresar un algoritmo. Se pueden utilizar para expresar la idea general del software del sistema embebido, aunque no precisamente la complejidad de su implementación.

Se utilizan prácticamente los mismos símbolos de un diagrama para un algoritmo. No siempre habrá un símbolo de finalización, puesto que los sistemas embebidos suelen trabajar con un bucle infinito. Hay cinco (5) símbolos básicos que se utilizan para desarrollar diagramas de flujo, mostrados en la Figura 4.

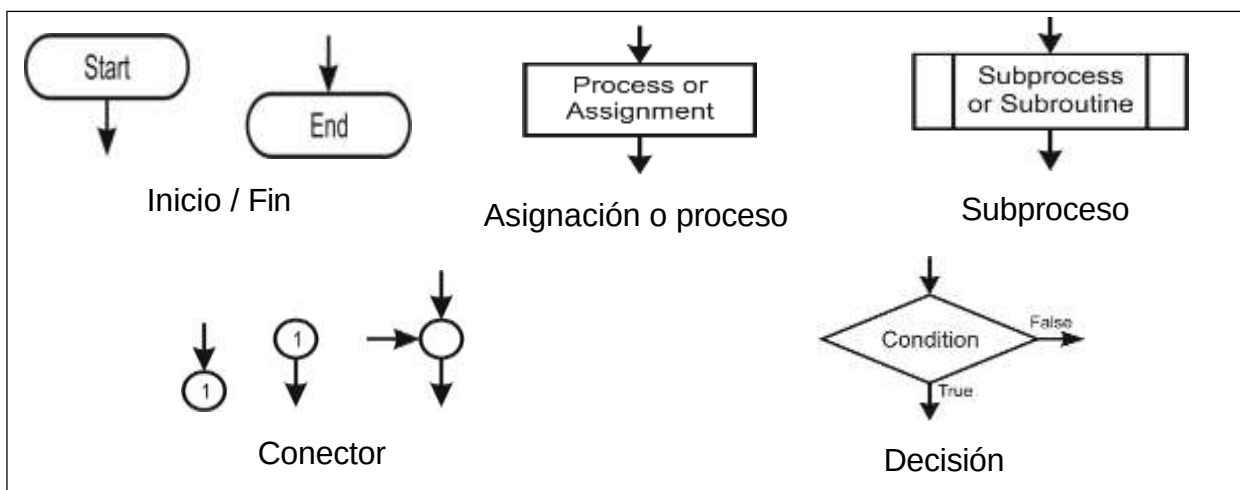


Figura 4. Símbolos básicos utilizados en los diagramas de flujo (Jiménez, Palomera, & Couvertier, 2014).

El símbolo de **Inicio/Fin** consiste en una forma ovalada. Como su nombre lo indica, el símbolo de inicio/finalización se utiliza para mostrar un punto de entrada al algoritmo o un punto de salida del algoritmo. Aunque no hay nada que impida hacer lo contrario, es una buena práctica de programación (altamente recomendada) especificar un único punto de entrada y también un único punto de salida. Un símbolo de inicio/finalización tiene solo una flecha, ya sea dejando un inicio o ingresando a un final.

El símbolo de **Asignación** o **Proceso** se dibuja como un rectángulo y dentro del rectángulo indicamos las asignaciones aritméticas o lógicas correspondientes, incluidas las expresiones. Podríamos incluir tantas declaraciones de asignación dentro del rectángulo como queramos, pero si comienza a verse abarrotado, entonces es una buena práctica dividirlo en varios rectángulos más pequeños que son más fáciles de ver. Los símbolos de proceso solo pueden tener una flecha de entrada y una flecha de salida.

El símbolo del **Subproceso** se parece mucho al símbolo de asignación ya que también es un rectángulo, pero incluye una línea vertical adicional tanto en el lado izquierdo como en el derecho. Se utiliza para indicar cada vez que se invoca una subrutina o llamada de función. Después de que el subproceso regresa, el algoritmo continúa con el siguiente símbolo en secuencia en el diagrama de flujo. Es una buena práctica usar

subprocesos en el desarrollo del algoritmo porque hace que el algoritmo sea menos tedioso, especialmente si el subproceso se llama varias veces. Por otro lado, debemos tener en cuenta que llamar a un subproceso y regresar de él agrega tiempo de ejecución a nuestro algoritmo. Al igual que los símbolos de proceso, los subprocesos tienen solo una flecha de entrada y una de salida.

El símbolo del **Conector** se usa cuando nos quedamos sin espacio en la página donde estamos desarrollando nuestro diagrama de flujo y necesitamos continuar en otra parte de la página o en una página completamente diferente. Los conectores pueden salir de una página, entrar en una nueva página, o indicar puntos de unión donde se fusionan múltiples flujos. Los símbolos de conector de página se usan en pares, ambos con el mismo número de identificación o letra adentro para indicar dónde continúa el flujo.

El símbolo de **Decisión** se asemeja a un diamante y, como su nombre lo indica, se utiliza para alterar el flujo del algoritmo en función del valor de alguna variable después de tomar una decisión en particular. La capacidad de tomar decisiones permite algoritmos potentes en lugar de los puramente secuenciales. Los bloques de decisión son los únicos que presentan dos salidas, una correspondiente a cada uno de los dos posibles resultados de decisión: verdadero o falso, sí o no, izquierda o derecha, etc.

Desarrollar un diagrama de flujo para un algoritmo involucra utilizar estos símbolos y conectarlos según sea necesario con flechas que indican la dirección del flujo (Jiménez, Palomera, & Couvertier, 2014).



Comprueba tu aprendizaje: En la Figura 5 se muestran combinaciones de estos símbolos, intente identificarlos.

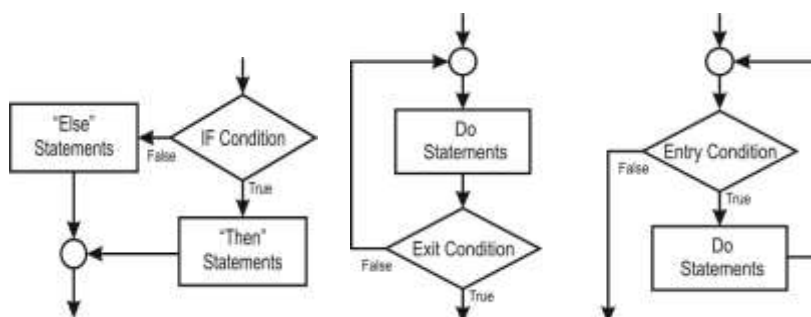


Figura 5. Combinación de símbolos de diagramas de flujo.

Las Figuras 6 y 7 muestran ejemplos de aplicación de diagramas de flujo. La Figura 6 muestra el diagrama de un programa que enciende y apaga un LED, con base en un

contador que decrementa el tiempo hasta llegar a cero. En este caso el programa no tiene un símbolo de Fin, por lo que se ejecuta indefinidamente.

En la Figura 7 se muestra un diagrama un poco más complejo. Consiste en la comunicación entre 2 nodos de un sistema de monitorización, e incluye 3 rutinas representadas con diagramas de flujo. Aquí además se ha representado el paso de datos entre ambos componentes mediante líneas segmentadas.

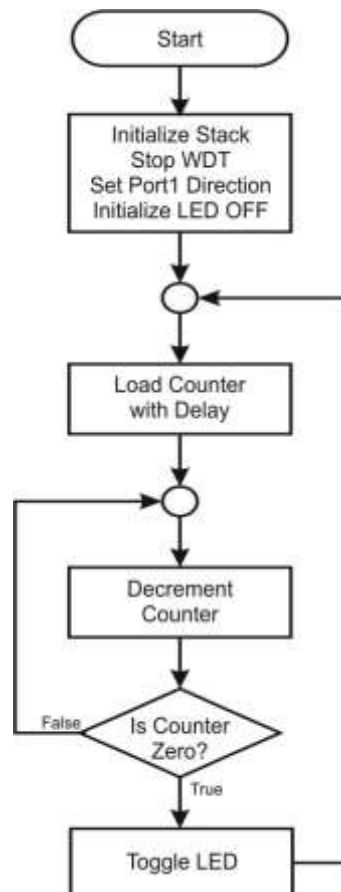


Figura 6. Diagrama de flujo para una aplicación sencilla (Jiménez, Palomera, & Couvertier, 2014).

Figura 7. Diagrama de flujo para un sistema de monitorización con dos componentes.

Programación del software.

Al desarrollar el código embebido hay 2 opciones:

- Programar en lenguaje ensamblador.
- Programar en lenguaje de alto nivel.

Aunque hoy en día es muy raro realizar la programación de un sistema embebido con lenguaje ensamblador, es posible encontrar proyectos que lo emplean. Se utiliza el lenguaje ensamblador cuando se aplica una o más de las siguientes razones:

- Es imperativo o muy deseable que los recursos se manejen de manera eficiente.
- Existe un conocimiento adecuado del lenguaje ensamblador y de la arquitectura particular.
- Política de la empresa o requisitos del usuario.

Al programar en lenguaje de alto nivel principalmente se utiliza el lenguaje C o un subconjunto de este lenguaje, aunque también es posible utilizar otros lenguajes como Python o C#. La elección del lenguaje suele depender del IDE utilizado, y además es necesario tener el compilador adecuado. En general, se utiliza el lenguaje de alto nivel cuando se aplica una o más de las siguientes razones:

- No es necesario un control estricto de los recursos.
- La eficiencia en el uso de recursos como la memoria y el poder no es una preocupación.
- Facilidad de reutilización del software.
- Política de la empresa o requisitos del usuario.
- Desconocimiento del lenguaje ensamblador y de la arquitectura particular.

La programación de software embebido es principalmente de tipo imperativa, ya se asemeja mucho en su estructura a un programa común en este paradigma. Entre otros aspectos, el desarrollo con lenguajes de alto nivel implica que:

- ✓ Se pueden incluir librerías
- ✓ Se definen constantes o expresiones, así como variables
- ✓ Hay tipos comunes y tipos más específicos para S.E.
- ✓ Es posible utilizar subrutinas o funciones, así como tipos de estructuras.
- ✓ No se suelen emplear punteros como tal, pero si se utilizan ocasionalmente direcciones de memoria propias de cada microcontrolador.

3.1.2 Integración del Software embebido

Aunque la forma de programar un Sistema Embebido no difiere mucho de la programación tradicional de tipo imperativa, hay que recordar que cada plataforma de hardware de destino es única. Por ejemplo, el método para comunicarse a través de una interfaz en serie puede variar de un MCU a otro y de una plataforma a otra. Debido a esto, hay que entender adecuadamente el proceso de desarrollo del software de un sistema embebido.

El proceso de construcción (*Build*).

Una vez codificado el software, el proceso de convertir la representación del código fuente de su software integrado en una imagen binaria ejecutable implica tres pasos distintos:

1. Cada uno de los archivos de origen debe compilarse o ensamblarse en un **archivo de objeto**.

2. Todos los archivos de objeto que resultan del primer paso deben estar vinculados para producir un solo archivo de objeto, llamado **programa reubicable** (*relocatable*).
3. Las direcciones de memoria física deben asignarse a las compensaciones relativas dentro del programa reubicable en un proceso llamado **reubicación** (*relocation*).

El resultado del paso final es un archivo que contiene una imagen binaria ejecutable que está lista para ejecutarse en el sistema embebido.

El proceso de desarrollo de software embebido que se acaba de describir se ilustra en la Figura 8. En esta figura, los tres pasos se muestran de arriba a abajo, con las herramientas que realizan los pasos que se muestran en cuadros que tienen esquinas redondeadas. Cada una de estas herramientas de desarrollo toma uno o más archivos como entrada y produce un solo archivo de salida.

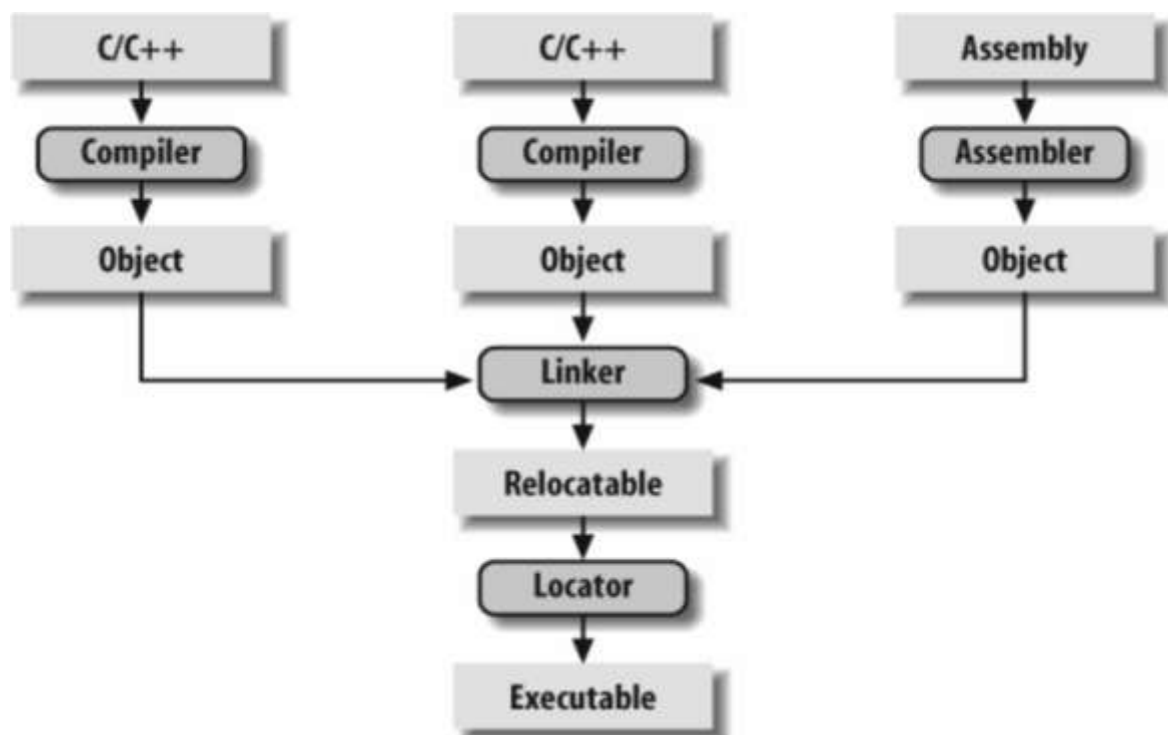


Figura 8. Esquema del proceso de desarrollo de software embebido (Barr & Massa, 2013).

Las herramientas que hacen posible este proceso normalmente están integradas en un IDE (Entorno Integrado de Desarrollo). En este caso esas herramientas son:

- El compilador (*compiler*): Convierte el código de alto nivel en un código propio de un procesador o microcontrolador. Crea un objeto por cada archivo de código de alto nivel. Ejemplo: GNU C Compiler gcc.
- El enlazador (*linker*): Combina los archivos de objeto en un archivo único. Ejemplo: GNU linker ld
- El ubicador (*locator*): Realiza la conversión de un programa reubicable a una imagen binaria ejecutable.

Además es necesario tener una herramienta que permita copiar la imagen binaria al espacio de memoria del microcontrolador. Esta herramienta depende del MCU.

Cada uno de los pasos del proceso de creación de software integrado es una transformación realizada por el software que se ejecuta en una computadora de propósito general. A esta computadora de desarrollo (generalmente una PC) se la denomina computadora host. El compilador, el ensamblador, el enlazador y el ubicador se ejecutan en una computadora central y no en el propio sistema integrado. Sin embargo, estas herramientas combinan sus esfuerzos para producir una imagen binaria ejecutable que se ejecutará correctamente solo en el sistema integrado de destino. La Figura 9 ilustra esta relación de tareas o responsabilidades.

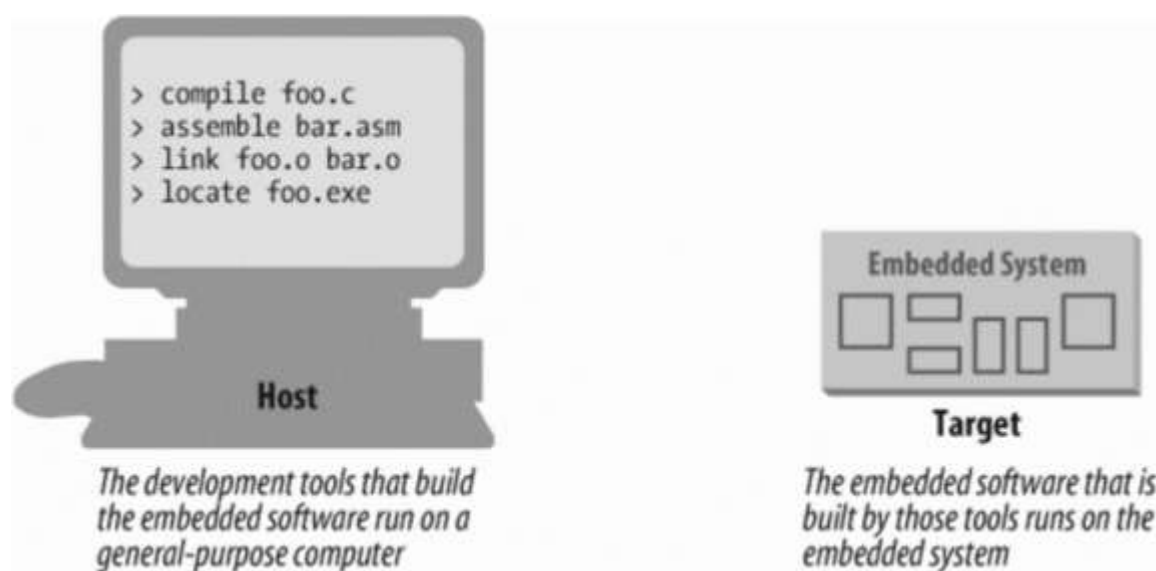


Figura 9. División de responsabilidades en el desarrollo del software embebido (Barr & Massa, 2013).

Compilación

El trabajo de un compilador es principalmente **traducir** programas escritos en algún lenguaje legible por humanos a un conjunto equivalente de códigos de operación para

un procesador en particular. En ese sentido, un ensamblador también es un compilador (podría llamarlo un "compilador de lenguaje ensamblador"), pero uno que realiza una traducción uno a uno mucho más simple de una línea de mnemónicos legibles por humanos al código de operación equivalente. Todo en esta sección se aplica por igual a compiladores y ensambladores. Juntas, estas herramientas constituyen el primer paso del proceso de creación de software integrado.

Por supuesto, cada procesador tiene su propio lenguaje de máquina único, por lo que se debe elegir un compilador que produzca programas para su procesador de destino específico. En el caso de los sistemas embebidos, este compilador casi siempre se ejecuta en la computadora host. Simplemente no tiene sentido ejecutar el compilador en el propio sistema embebido. Un compilador como este, que se ejecuta en una plataforma informática y produce código para otra, se denomina compilador cruzado. El uso de un compilador cruzado es una de las características definitorias del desarrollo de software embebido.

Independientemente del lenguaje de entrada (C, C++, ensamblador o cualquier otro), la salida del compilador cruzado será un archivo de objeto. Este es un archivo binario con formato especial que contiene el conjunto de instrucciones y datos resultantes del proceso de traducción del idioma. Aunque partes de este archivo contienen código ejecutable, el archivo objeto no se puede ejecutar directamente. De hecho, la estructura interna de un archivo de objeto enfatiza lo incompleto del programa más grande.

Enlazado

Todos los archivos de objetos resultantes de la compilación en el paso uno deben combinarse. Los archivos de objetos en sí están incompletos individualmente, sobre todo porque algunas de las referencias de funciones y variables internas aún no se han resuelto. El objetivo del enlazador es combinar estos archivos de objetos y, en el proceso, resolver todos los símbolos no resueltos.

La salida del enlazador es un nuevo archivo de objeto que contiene todo el código y los datos de los archivos de objeto de entrada y tiene el mismo formato de archivo de objeto.

Mientras que el enlazador está en el proceso de fusionar los contenidos de la sección, también está buscando símbolos sin resolver. Por ejemplo, si un archivo de objeto

contiene una referencia no resuelta a una variable llamada *foo* y se declara una variable con ese mismo nombre en uno de los otros archivos de objeto, el enlazador los comparará. La referencia no resuelta se reemplazará con una referencia a la variable real. Por ejemplo, si *foo* está ubicado en el desplazamiento 14 de la sección de datos de salida, su entrada en la tabla de símbolos ahora contendrá esa dirección.

Una de las cosas que hacen automáticamente las herramientas tradicionales de desarrollo de software es insertar **código de inicio**: un pequeño bloque de código en lenguaje ensamblador que prepara el camino para la ejecución de software escrito en un lenguaje de alto nivel.

Ubicación

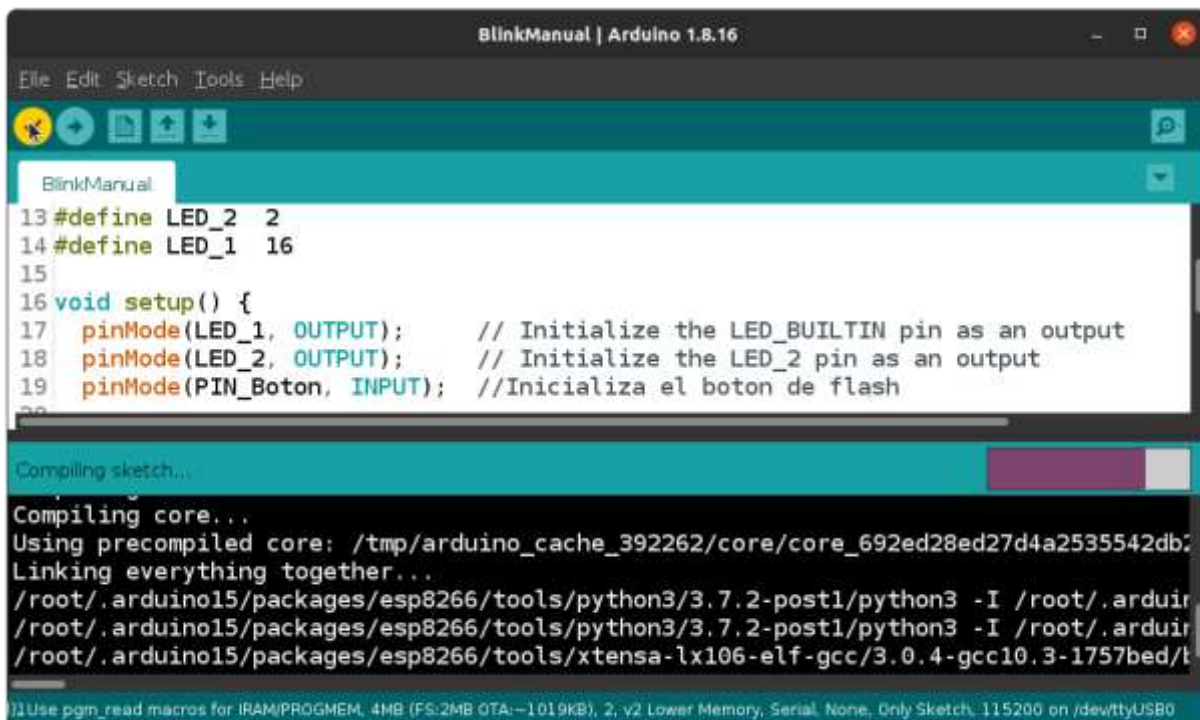
La herramienta que realiza la conversión de un programa reubicable a una imagen binaria ejecutable se denomina ubicador o localizador. Asume la responsabilidad del paso más fácil del proceso de creación. De hecho, el desarrollador debe hacer la mayor parte del trabajo en este paso, proporcionando información sobre la memoria de la placa de microcontrolador de destino como entrada al ubicador. El ubicador usa esta información para asignar direcciones de memoria física a cada una de las secciones de código y datos dentro del programa reubicable. Luego produce un archivo de salida que contiene una imagen de memoria binaria que se puede cargar en el objetivo.

Ya sea que esté escribiendo software para una computadora de propósito general o un sistema integrado, en algún momento las secciones de su programa reubicable deben tener direcciones reales asignadas. A veces, el software que ya está en el destino realiza esto.

En algunos casos, existe una herramienta de desarrollo separada, llamada localizador, para asignar direcciones; mientras que en otras esta función está integrada en el enlazador (Barr & Massa, 2013).

Ejemplo del proceso de integración con Arduino IDE.

A continuación veremos un ejemplo del proceso de integración de software de un sistema embebido, tomando como ejemplo el proceso que se da con Arduino IDE en un PC con el sistema operativo Ubuntu. Este ejemplo se ha realizado con un programa escrito para la placa de evaluación NodeMCU, que tiene un microcontrolador ESP8266. La Figura 10 muestra una captura del proceso de compilación y enlazado.



```

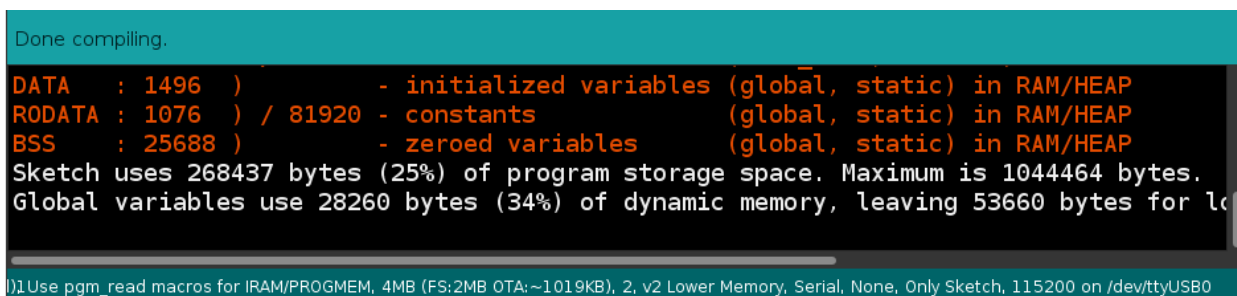
BlinkManual | Arduino 1.8.16
File Edit Sketch Tools Help

13 #define LED_2 2
14 #define LED_1 16
15
16 void setup() {
17   pinMode(LED_1, OUTPUT); // Initialize the LED_BUILTIN pin as an output
18   pinMode(LED_2, OUTPUT); // Initialize the LED_2 pin as an output
19   pinMode(PIN_Boton, INPUT); //Inicializa el boton de flash
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

Compiling sketch...
Compiling core...
Using precompiled core: /tmp/arduino_cache_392262/core/core_692ed28ed27d4a2535542db2
Linking everything together...
/root/.arduino15/packages/esp8266/tools/python3/3.7.2-post1/python3 -I /root/.ardui
/root/.arduino15/packages/esp8266/tools/python3/3.7.2-post1/python3 -I /root/.ardui
/root/.arduino15/packages/esp8266/tools/xtensa-lx106-elf-gcc/3.0.4-gcc10.3-1757bed/t
1) Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA:~1019KB), 2, v2 Lower Memory, Serial, None, Only Sketch, 115200 on /dev/ttyUSB0
    
```

Figura 10. Inicio del proceso de compilación y enlace de código en Arduino IDE.

Durante este proceso se generan los archivos objeto del código del programa, tal como se explicó antes. Al finalizar, el IDE muestra un resumen de la cantidad de memoria utilizada en relación al total disponible, tanto para el programa como para las variables que se utilizarán (Figura 11).



```

Done compiling.

DATA : 1496 ) - initialized variables (global, static) in RAM/HEAP
RODATA : 1076 ) / 81920 - constants (global, static) in RAM/HEAP
BSS : 25688 ) - zeroed variables (global, static) in RAM/HEAP
Sketch uses 268437 bytes (25%) of program storage space. Maximum is 1044464 bytes.
Global variables use 28260 bytes (34%) of dynamic memory, leaving 53660 bytes for local variables.

1) Use pgm_read macros for IRAM/PROGMEM, 4MB (FS:2MB OTA:~1019KB), 2, v2 Lower Memory, Serial, None, Only Sketch, 115200 on /dev/ttyUSB0
    
```

Figura 11. Resultado de la compilación y enlazado.

Los archivos producto de la compilación y enlazado no se generan en el mismo directorio del código fuente, sino que en otro directorio aparte, que puede ser temporal. Es necesario revisar el reporte de la compilación para saber la ubicación exacta de estos. En la Figura 12 se muestra el listado de los archivos generados en la proceso descrito en las figuras anteriores.

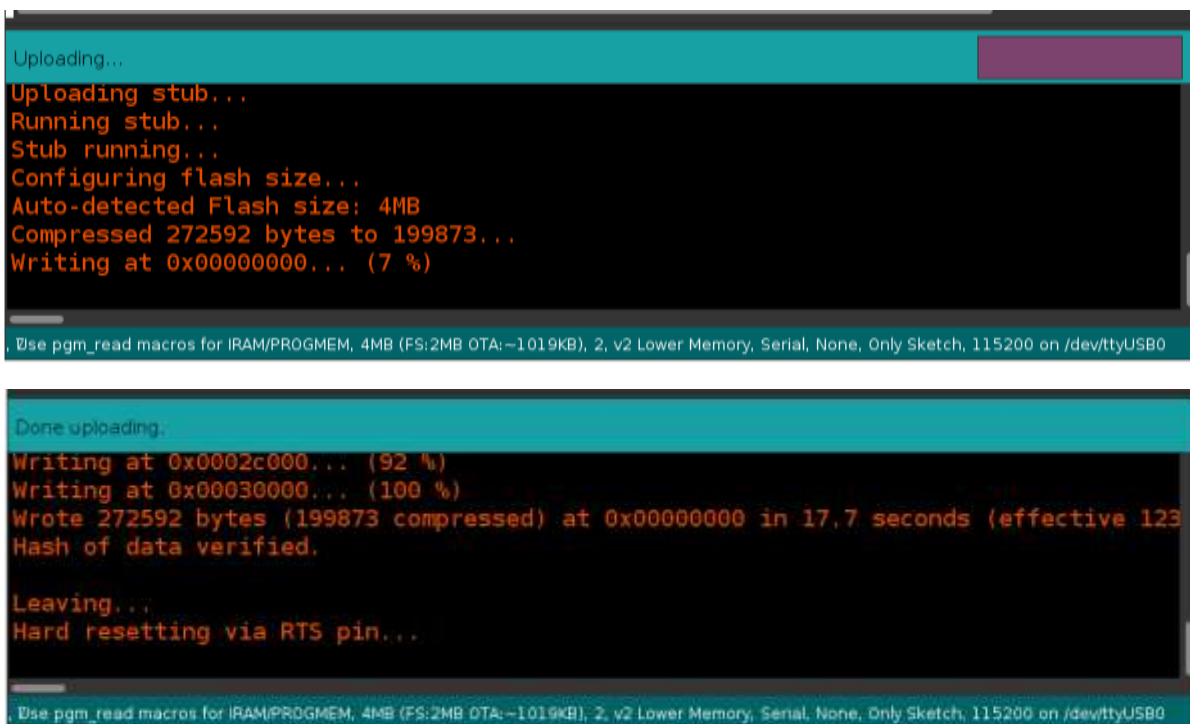
```

mnaviam@Lenovo2:~$ ls -l /tmp/arduino_build_484685
total 4024
-rw-r--r-- 1 root root 272592 jul  8 11:04 BlinkManual.ino.bin
-rwxr-xr-x 1 root root 2046152 jul  8 11:04 BlinkManual.ino.elf
-rw-r--r-- 1 root root 1741367 jul  8 11:04 BlinkManual.ino.map
-rw-r--r-- 1 root root 1748 jul  8 11:04 build.options.json
drwxr-xr-x 5 root root 12288 jul  8 11:02 core
-rw-r--r-- 1 root root 419 jul  8 11:04 includes.cache
drwxr-xr-x 2 root root 4096 jul  8 11:02 ld_h
drwxr-xr-x 2 root root 4096 jul  8 11:02 libraries
-rw-r--r-- 1 root root 9096 jul  8 11:04 local.eagle.app.v6.common.ld
-rw-r--r-- 1 root root 858 jul  8 11:04 local.eagle.flash.ld
drwxr-xr-x 2 root root 4096 jul  8 11:02 preproc
drwxr-xr-x 2 root root 4096 jul  8 11:04 sketch
    
```

Figura 12. Archivos generados en el proceso de compilación y enlazado.

El archivo ejecutable obtenido al finalizar el proceso de Ubicación o localización, debe ser cargado o grabado en la memoria no volátil del microcontrolador (puede ser de tipo flash, EEPROM, u otra similar). La Figura 13 muestra una captura de pantalla de este proceso. Al finalizar, aparecerá un mensaje de que el proceso terminó, y que el microcontrolador se ha reiniciado con el nuevo código; o un mensaje de error si algo no ha salido bien.

Cabe indicar que si hubo algún problema en la compilación, enlazado o ubicación, no se generará un programa ejecutable adecuado, y por lo tanto no se podrá realizar este último paso del desarrollo del software embebido.



```

Uploading...
Uploading stub...
Running stub...
Stub running...
Configuring flash size...
Auto-detected Flash size: 4MB
Compressed 272592 bytes to 199873...
Writing at 0x00000000... (7 %)

Done uploading.
Writing at 0x0002c000... (92 %)
Writing at 0x00030000... (100 %)
Wrote 272592 bytes (199873 compressed) at 0x00000000 in 17.7 seconds (effective 123
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
    
```

Figura 13. Proceso de ubicación o carga del software en la placa NodeMCU.



Recuerda que: Para realizar un adecuado proceso de compilación, enlazado, ubicación y carga del programa embebido, se debe especificar bien con que microcontrolador se está trabajando.



Tema 3.2: Sistemas operativos para dispositivos embebidos

3.2.1 Sistemas operativos embebidos

Un sistema operativo embebido es un sistema operativo que se ejecuta sobre un sistema embebido. Generalmente se ejecutan sobre dispositivos que difieren de un computador común (televisores, hornos microondas, dispositivos móviles, entre otros)

Usualmente tienen algunas características de sistemas de tiempo real, pero a la vez tienen restricciones de tamaño, memoria y energía que los hacen especiales. En algunos de casos, los diseños se desarrollan en torno a sistemas operativos en tiempo real (*Real-Time Operating Systems RTOS*) (Perez, 2009).

La estructura del software embebido, cuando se diseña o implementa como un sistema operativo embebido, suele tener estos 3 elementos (Figura 14):

- Tareas (subrutinas): El software de la aplicación embebida se puede dividir en subprogramas o tareas.
- Núcleo del sistema (kernel): Es el componente del software que maneja los recursos del sistema embebido. Recibe peticiones de las tareas y brinda comunicación entre ellas
- Servicios (drivers): Son piezas de código que dan funcionalidad al recurso del sistema. Se los conoce como drivers (Jiménez, Palomera, & Couvertier, 2014).

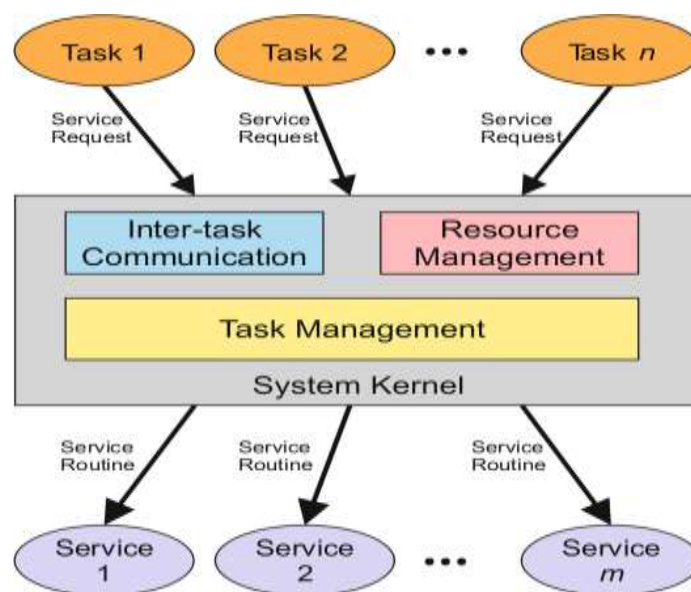


Figura 14. Elementos de un sistema operativo embebido.

Sistemas operativos en tiempo real.

Los sistemas en tiempo real son capaces de gobernar varios periféricos en un determinado tiempo. Son utilizados principalmente en sistemas embebidos.

Un RTOS es un sistema operativo multitarea destinado para aplicaciones en tiempo real, pero que tiene un funcionamiento más o menos predeterminado. Mientras que en un sistema operativo de propósito general se pueden ejecutar varias tareas en “paralelo”, mediante un planificador de tareas; en un RTOS el planificador está diseñado para proporcionar un patrón de ejecución predecible (normalmente descrito como **determinista**).

Entre algunas características de los RTOS tenemos que:

- Debe soportar un método de programación que garantice el tiempo de respuesta, especialmente a tareas críticas
- Se debe poder dar prioridad a las tareas
- Tiene que admitir mecanismos de sincronización de tareas predecibles.
- Administra los recursos de hardware y software.
- Determinista: garantiza la finalización de la tarea en un plazo establecido.
- Deben conocerse y minimizarse las limitaciones de tiempo del comportamiento.
 - Latencia de interrupción.
 - Tiempo mínimo de cambio de tarea (cambio de contexto).

A continuación tenemos algunos ejemplos de RTOS:

- **FreeRTOS**: Es pequeño y simple (el núcleo en sí consta de solo tres archivos implementados en C), con soporte para múltiples plataformas (www.freertos.org).
- **QNX**: RTOS basado en Unix, con aplicaciones como dispositivos móviles, consolas, y autos (<https://blackberry.qnx.com/>).
- **VxWorks**: Sistema propietario, con reconocida estabilidad y seguridad certificada. Aplicaciones desde cámaras o impresoras, hasta aviones o robots como el Curiosity (www.windriver.com/).
- **RTLinux**: Un RTOS con kernel Linux y ejecución determinista. Adquirido y luego discontinuado por Wind River, para ofertar su propio RTOS Wind River Linux (www.windriver.com/products/linux).

Además, existen otros sistemas operativos embebidos que no son precisamente de tiempo real, pero igualmente se los utiliza en sistemas embebidos. Entre algunos podemos mencionar:

- **TinyOS:** Es muy utilizado en sistemas embebidos, especialmente para redes de sensores inalámbricas. Está escrito en el lenguaje nesC. (<http://www.tinyos.net/>)
- **RIOT:** Es un S.O. gratuito de código abierto, que soporta varias arquitecturas y dispositivos de bajo consumo, y permite programar aplicaciones en C y C++ (<https://www.riot-os.org/>).
- **Conkiti OS:** Está orientado a dispositivos con restricciones de energía, especialmente para IoT. Integra multitasking, TCP/IP, e incluso una interfaz gráfica básica (<http://www.contiki-os.org/>).
- **eCos** (*embedded configurable operating system*): Funciona sobre varias arquitecturas. Desarrollado para ser altamente configurable (más de 200 puntos de configuración a nivel código fuente) (<http://ecos.sourceware.org/>).

Caso de estudio de un RTOS: RIOT.

RIOT es un sistema operativo de código abierto basado en microkernel, diseñado para cumplir con los requisitos de los dispositivos de Internet de las cosas (IoT) y otros dispositivos embebidos. Los requisitos que se cumplen incluyen:

- Muy bajo requerimiento de memoria (del orden de unos pocos kilobytes).
- Alta eficiencia energética.
- Capacidades en tiempo real.
- Soporte para una amplia gama de hardware de bajo consumo.
- Pilas de comunicación para redes inalámbricas y redes cableadas.

Además hay utilidades que incluyen bibliotecas criptográficas, estructuras de datos (tablas hash, colas de prioridad), un shell, y más. El código base de RIOT está estructurado en cinco grupos principales.

- El núcleo (*core*)
- Código específico de la plataforma (*cpu; boards*)
- Controladores de dispositivos (*drivers*)
- Bibliotecas y código de red (*sys; pkg*)
- Aplicaciones para demostración de funcionalidades y para pruebas (*examples; tests*)

Además, RIOT incluye una colección de scripts para varias tareas (*dist*), así como un entorno predefinido para generar esta documentación (*doc*). La Figura 15 muestra el esquema de estos componentes. Se puede ver que un grupo es independiente del hardware, mientras que otros (nivel inferior) son dependientes del hardware donde se implemente.

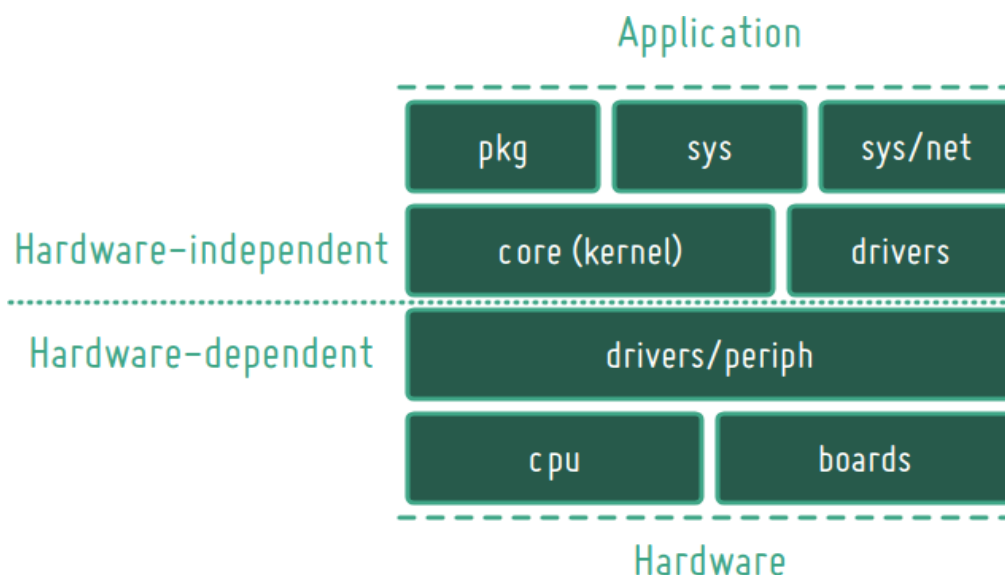


Figura 15. Esquema de los componentes de RIOT.

Veamos la descripción de los principales componentes de este sistema operativo:

- **core:** Este directorio contiene el núcleo real. Consiste en el planificador, la comunicación entre procesos (mensajes), los subprocesos y la sincronización de subprocesos, así como las estructuras de datos compatibles y las definiciones de tipos.
- **boards:** Contiene la configuración específica para la CPU que contiene la placa. Incluye principalmente la configuración de periféricos y mapeo de pines, configuración de los dispositivos integrados, y configuración del reloj de CPU. También puede incluir algunos scripts y archivos de configuración necesarios para interactuar con la placa.
- **cpu:** Para cada CPU compatible, este directorio contiene un subdirectorio con el nombre de la CPU. Estos directorios luego contienen todas las configuraciones específicas de la CPU, como implementaciones de administración de energía, manejo de interrupciones y vectores, entre otros.

- En el subdirectorio *periph* de cada CPU, puede encontrar las implementaciones de los controladores periféricos de la CPU como SPI, UART, GPIO, etc.
- **drivers**: Contiene los controladores para dispositivos externos, como interfaces de red, sensores y actuadores. Cada controlador de dispositivo se coloca en su propio subdirectorio con el nombre de ese dispositivo.
- **sys**: Todos los módulos que no forman parte de la abstracción de hardware ni de los controladores de dispositivos se pueden encontrar en este directorio. Las bibliotecas incluyen estructuras de datos (p. ej., bloom, color), bibliotecas criptográficas (p. ej., hash, AES), API de alto nivel, entre otras.
- **sys/net**: Es donde reside todo el código de red en RIOT. Aquí se pueden encontrar las implementaciones de la pila de red.
- **pkg**: Incluye bibliotecas externas, que pueden ser parchadas para funcionar en RIOT (RIOT, 2022).



Dato útil: En la web de RIOT se puede encontrar tanto los CPUs que soporta este S.O. (<https://www.riot-os.org/cpus.html>) como las placas de evaluación a las que ha sido adaptado (<https://www.riot-os.org/boards.html>).

3.2.2 Drivers y mecanismos de interrupción

Fundamentos de interrupciones.

Una **interrupción** (del inglés *interrupt request*) es una señal recibida por un procesador, que indica que debe “interrumpir” el curso de ejecución actual y pasar a ejecutar código específico para tratar esta situación. Es una suspensión temporal de la ejecución de un proceso, para pasar a ejecutar una subrutina que no forma parte de un programa principal (como se ve en la Figura 16). En un sistema embebido también pueden producirse interrupciones.



Figura 16. Esquema de funcionamiento de una interrupción.

De acuerdo a la fuente donde se produce la misma, las interrupciones pueden ser:

- **Interrupciones de hardware:** Estas son asíncronas y externas a la ejecución del procesador (por lo general provocadas por dispositivos de E/S)
- **Excepciones:** Son aquellas que se producen de forma síncrona a la ejecución del procesador y por tanto podrían predecirse si se analiza la traza del programa ejecutado (desbordamiento de una variable, división para 0, etc.)
- **Interrupciones por software:** Son aquellas generadas por un programa en ejecución. existen distintas instrucciones en el código máquina que permiten al programador producir una interrupción.

Las interrupciones son útiles para hacer que las cosas sucedan automáticamente, y pueden ayudar a resolver problemas de temporización. Con las interrupciones, no se necesita verificar constantemente el valor de un pin. Cuando se detecta un cambio, se activa un evento que llama a una función. Esta función se llama **rutina de servicio de interrupción** (ISR).

Además de la ventaja de la eficiencia del uso de la CPU, el uso del servicio de interrupciones también brinda las siguientes ventajas:

- ✓ Código compacto y modular: las ISR inducen la modularidad del código y la reutilización del software.
- ✓ Consumo de energía reducido: dado que los ISR conducen a menos ciclos de CPU, esto tiene un impacto directo en la cantidad de energía consumida por la aplicación, en particular cuando se combina con modos de bajo consumo.
- ✓ Tiempo de respuesta más rápido: cuando es necesario atender muchos eventos y dispositivos, los ISR bien diseñados brindan una respuesta rápida al evento desencadenante. Las prioridades bien coordinadas entre las fuentes permitirán una respuesta rápida con una mínima sobrecarga de procesamiento (Jiménez, Palomera, & Couvertier, 2014).

Implementación de Interrupciones.

La mayoría de microcontroladores que se pueden programar en Arduino IDE, incluyendo el NodeMCU, soportan tanto interrupciones de hardware como de software. Veamos ambos casos.

Para configurar una interrupción de hardware relacionada a un GPIO, se utiliza la función **`attachInterrupt()`**, que acepta como argumentos: el pin de interrupción GPIO, el nombre de la función a ejecutar, y el modo:

```
attachInterrupt(digitalPinToInterrupt(GPIO), ISR, mode);
```

El detalle de los argumentos de la función `attachInterrupt()` son:

- **GPIO interrupt pin:** Es el pin que se va a verificar si cambia de valor o señal.
- **ISR:** Es el nombre de la función que se llamará cada vez que se active la interrupción. No debe llevar los paréntesis o necesitar argumentos.
- **Modo:** Indica el tipo de cambio que producirá la interrupción. Hay 4 modos, aunque no todos los microcontroladores los soportan todos:
 - *LOW*: cuando pasa a nivel bajo.
 - *CHANGE*: cuando cambia de valor (sea a alto o a bajo).
 - *FALLING*: durante el flanco de bajada de señal.
 - *RISING*: durante el flanco de subida de señal.

Para las interrupciones es posible también utilizar *timers* (temporizadores). Un *timer* es un elemento que permite contar un periodo de tiempo. Se utiliza para controlar la ejecución de una tarea, subrutina o evento. También se denominan **interrupciones programadas**, debido a que permiten establecer que una interrupción o tarea se ejecute después de un determinado tiempo.

Los *timers* son principalmente de hardware (pueden venir uno o más integrados en el microcontrolador), pero también es posible implementar *timers* software.

En Arduino IDE es posible implementar temporizadores de varias maneras. Principalmente se usan las instrucciones *delay()* y *millis()*:

- ***delay()***: detiene la ejecución del programa durante *n* milisegundos, donde *n* es un número entero largo pasado como argumento.

```
delay(time_in_milliseconds);
```

- ***millis()***: devuelve el número de milisegundos que han pasado desde que el programa empezó a ejecutarse.

```
millis();
```

En el primer caso se trata de una instrucción bloqueante (el programa no continúa con el resto de su ejecución hasta que el temporizador termine); mientras que la segunda no lo es, es decir que el programa sigue con la siguiente instrucción.

Para programar una interrupción por software, utilizando temporizadores, se puede utilizar un objeto **Ticker** (incluido en la librería *Ticker.h*), que permite definir un temporizador, que a su vez llama a una función que se le indique. Por ejemplo se puede añadir una función al temporizador especificando su tiempo en milisegundos:

```
Ticker.attach_ms_scheduled(10000, IRS_function);
```

O bien su el tiempo para ejecutar la interrupción en segundos

```
Ticker.attach(10, IRS_function);
```

En ambos casos, cada 10 segundos se ejecutará la función *IRS_function*. La única diferencia, es que en el primer caso solo se ejecuta una vez, mientras que en el segundo se repetirá la ejecución cada 10 segundos (por el modo *scheduled*).

Además, existen también librerías externas como TimerOne (disponible en <https://www.prometec.net/wp-content/uploads/2014/11/TimerOne.zip>) que permiten implementar interrupciones con temporizadores. Cuando se quiera utilizar esta librería hay que importarla:

```
#include <TimerOne.h>
```

Esto nos crea un objeto llamado *Timer1* directamente. Luego se programa el intervalo de disparo de la interrupción en microsegundos (1/1000 milisegundos):

```
Timer1.initialize(150000); // 150 ms
```

Y por último se indica el ISR de la interrupción:

```
Timer1.attachInterrupt(ISR_Callback);
```

Antes de utilizar este tipo de librerías es recomendable (y necesario) consultar para que placa de microcontrolador están diseñadas.



Recuerda que: Al igual que en las interrupciones por hardware, en las interrupciones por software las funciones ISR no deben llevar argumentos.

3.2.3 Depuración del SW embebido

El ciclo de desarrollo de software comienza con el diseño y la primera implementación del código. Después de eso, generalmente hay iteraciones de las etapas de compilación, descarga y depuración, y corrección de errores (Figura 17). Estas tres últimas etapas pero también pueden llevar mucho tiempo, por lo que existen herramientas disponibles que nos pueden ayudar. Existen opciones en línea, pero principalmente para sistemas operativos de escritorio.

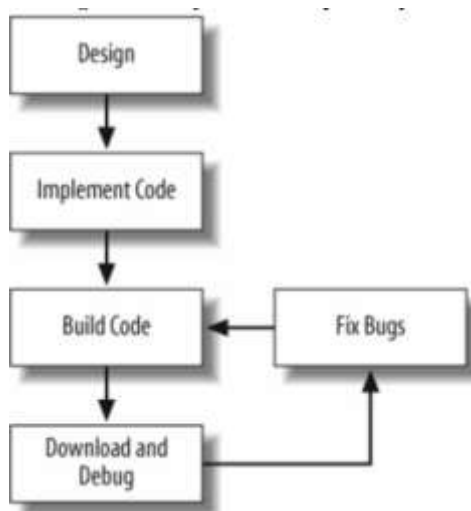


Figura 17. Esquema del proceso de depuración del software.

Debido a que el esquema de la Figura 17 es un diagrama muy básico, otras etapas que pueden ser necesarias son la creación de perfiles y la optimización. La creación de perfiles permite a un desarrollador determinar varias métricas sobre un programa, como dónde pasa la mayor parte del tiempo el procesador. La optimización es el proceso mediante el cual el desarrollador trata de eliminar los cuellos de botella en el software utilizando diversas técnicas, como la implementación de código de tiempo crítico en lenguaje ensamblador. Muy a menudo, las técnicas de optimización son específicas del compilador, del procesador o del sistema.

Una vez que se completa el ciclo de desarrollo, normalmente se realizan pruebas a nivel del sistema. Y después de que se envía un producto, el software entra en su fase de mantenimiento durante el ciclo de vida del producto, cuando el código debe recibir soporte y mantenimiento. Las técnicas de depuración existentes también se pueden aplicar a la etapa de mantenimiento.

Debido a que el código debe probarse repetidamente en el hardware de destino, lo ideal es un método rápido y sencillo para cargar software en el destino y verificar como funciona en este.

Un **monitor de depuración**, también llamado monitor de ROM, es un pequeño programa que reside en la memoria no volátil del hardware de destino que facilita varias operaciones necesarias durante el desarrollo. Una de las tareas que maneja un monitor de depuración es la inicialización básica del hardware. Un monitor de depuración le permite descargar y ejecutar software en la RAM para depurar el programa.

La mayoría de los monitores de depuración incorporan algún tipo de interfaz de línea de comandos (CLI) donde los comandos se pueden enviar al monitor de depuración para su ejecución en el hardware de destino, por ejemplo, a través del puerto serie. Estos comandos incluyen descargar software, ejecutar el programa, mirar (leer) y tocar (escribir) la memoria y los registros del procesador, comparar o mostrar bloques de memoria y establecer configuraciones de inicialización para el hardware (Barr & Massa, 2013).

Algunas placas de microcontroladores incluyen también algún chip adicional que permite la depuración (incluso instrucción a instrucción) del funcionamiento de un programa. En el caso de la placa NodeMCU que utilizamos para prácticas, no trae ninguno. Pero por ejemplo, las placas de evaluación STM32-Nucleo-64 (que traen microcontroladores tipo Cortex-M), vienen con un chip denominado ST-LINK que permite tanto programar como depurar el funcionamiento del microcontrolador.



Recuerda que: Una forma sencilla de depuración es enviar mensajes por un puerto serial (UART) al computador cada vez que se ejecutan ciertas subrutinas o tareas, o cuando se produce una interrupción que se haya programado, y leer estos mensajes en el monitor serie.



Video: En este video podemos ver como instalar el entorno de desarrollo de RIOT OS para poder instalar este sistema en una placa con un microcontrolador ESP32, de la misma familia que el ESP8266 <https://www.youtube.com/watch?v=8x0ZjLOxB7s>.



Herramientas & Software de la asignatura

- **Arduino IDE**, entorno de programación para sistemas embebidos. Disponible para descargar en <https://www.arduino.cc/en/software>.
- **Circuito.io**, entorno en línea para diseñar circuitos con componentes. Disponible en <https://www.circuito.io/>.
- **Tinkercad**, simulador en línea para modelar y simular circuitos electrónicos. Disponible en <https://www.tinkercad.com/>.

Bibliografía

Barr, M., & Massa, A. (2013). *Programming Embedded Systems*. O'Reilly.

Jiménez, M., Palomera, R., & Couvertier, I. (2014). *Introduction to Embedded Systems*. Springer.

Perez, D. (2009). Sistemas Embebidos y Sistemas Operativos Embebidos. *Lecturas en Ciencias de la Computación*.

RIOT. (2022). *RIOR Documentation*. Recuperado el 05 de 2022, de <https://doc.riot-os.org/>

Este compendio recoge textualmente documentos e información de varias fuentes debidamente citadas, así como referencias elaboradas por el autor para conectar los diferentes temas.

Se lo utiliza únicamente con fines educativos.